

create the same structure for spark MLIB

Here is the step-by-step execution guide for **Spark MLlib Machine Learning (without Hadoop)**, formatted with system/use-case placeholders enclosed in `<< >>`.

TXT

Group A: Environment Setup & Data Preparation

Terminal 1: Setup Python Virtual Environment & Input Files

1. Verify Python & Java Installations:

Bash



```
python3 --version
pip3 --version
java -version
```

1. Set Up Java Environment Variables (if needed):

Bash



```
export JAVA_HOME=<< /usr/lib/jvm/java-17-openjdk-amd64 >>[cite: 1]
export PATH=$JAVA_HOME/bin:$PATH[cite: 1]
```

1. Create & Activate PySpark Virtual Environment:

Bash



```
python3 -m venv ~/<<spark-venv>>[cite: 1]
source ~/<<spark-venv>>/bin/activate[cite: 1]
```

```
pip install pyspark==<<3.5.7>> numpy[cite: 1]  
pyspark --version[cite: 1]
```

1. Create Working & Input Directories:

Bash



```
mkdir -p ~/<<spark-mllib-lab>>/<<input>>[cite: 1]  
cd ~/<<spark-mllib-lab>>[cite: 1]
```

1. Create Sample CSV Dataset:

Bash



```
nano <<input/student_hours.csv>>[cite: 1]
```

Paste Content:

[cite: 1]

Code snippet



```
<<hours>>,<<result>>  
1,0  
2,0  
3,0  
4,1  
5,1  
6,1  
7,1  
8,1
```

1. Verify Input Data File:

Bash



```
cat <<input/student_hours.csv>>[cite: 1]
```

Group B: PySpark MLlib Standalone Script Execution

Terminal 1: Script Execution

1. Create PySpark MLlib Script:

Bash



```
nano <<student_pass_prediction.py>>[cite: 1]
```

Paste Content:

[cite: 1]

Python



```
from pyspark.sql import SparkSession
from pyspark.ml.feature import VectorAssembler
from pyspark.ml.classification import LogisticRegression

# Configure Spark Session to run locally without Hadoop / HDFS
spark = SparkSession.builder \
    .appName("<<StudentPassPrediction>>") \
    .master("<<local[*]>>") \
    .config("spark.hadoop.fs.defaultFS", "file:///") \
    .getOrCreate()

# Load local CSV dataset
df = spark.read.csv(
    "file:///home/<<ubuntu>>/<<spark-mllib-lab>>/<<input/student_hours.csv>>",
    header=True,
    inferSchema=True
)

print("\n--- Training Data ---")
df.show()
```

```
# Step 1: Feature Engineering – Convert hours into ML features vector
assembler = VectorAssembler(
    inputCols=["hours"],
    outputCol="features"
)
df_features = assembler.transform(df)

# Step 2: Initialize Logistic Regression Model
lr = LogisticRegression(
    featuresCol="features",
    labelCol="result"
)

# Step 3: Train Model
model = lr.fit(df_features)

# Step 4: Make Prediction for a student studying 5 hours
test_df = spark.createDataFrame([(5,)], ["hours"])
test_features = assembler.transform(test_df)

print("\n--- Prediction Output ---")
predictions = model.transform(test_features)
predictions.select("hours", "prediction").show()

# Stop Spark Session
spark.stop()
print("\nSpark MLlib Pipeline Completed Successfully.")
```

1. Run PySpark MLlib Job via `spark-submit`:

Bash



```
spark-submit <<student_pass_prediction.py>>[cite: 1]
```

Expected Output:

[cite: 1]

Plaintext



```
+-----+-----+
|hours|prediction|
+-----+-----+
|    5|        1.0|
+-----+-----+
```

Group C: Interactive PySpark Shell Option (Optional)

Terminal 1: Interactive Shell Execution

1. Launch PySpark Shell:

Bash



```
pyspark[cite: 1]
```

1. Run MLlib Pipeline Interactively:

Python



```
from pyspark.ml.feature import VectorAssembler[cite: 1]
from pyspark.ml.classification import LogisticRegression[cite: 1]

# Create small dataset directly in shell
data = [(1, 0), (2, 0), (3, 0), (4, 1), (5, 1), (6, 1), (7, 1), (8, 1)][cite: 1]
df = spark.createDataFrame(data, ["hours", "result"])[cite: 1]

# Feature transformation
assembler = VectorAssembler(inputCols=["hours"], outputCol="features")[cite: 1]
df_transformed = assembler.transform(df)[cite: 1]

# Model training
lr = LogisticRegression(featuresCol="features", labelCol="result")[cite: 1]
model = lr.fit(df_transformed)[cite: 1]

# Test prediction
test = spark.createDataFrame([(5,)], ["hours"])[cite: 1]
test_transformed = assembler.transform(test)[cite: 1]
model.transform(test_transformed).select("hours", "prediction").show()[cite: 1]
```

